

Mtech Software Engineering

Software Architecture - Assignment #1



Objective: To get familiar with the software architecture

System Name: LogiClear[OCR-logistics]: Automated Freight Document Processing Platform.

Salandri Shirisha - 2025TM93131 - 2025tm93131@wilp.bits-pilani.ac.in

PURPOSE OF THE SYSTEM



System Name

LogiClear[OCR-logistics] : Automated Freight Document Processing Platform.

Core Goal: To automate the extraction of critical data from a diverse set of shipping and customs documents, thereby accelerating freight clearance, reducing manual errors, and providing real-time visibility into the logistics chain.

Key Business Drivers:

- **Reduce Demurrage & Detention Costs:** Minimize costly delays at ports and warehouses caused by slow manual paperwork processing.
- **Improve Data Accuracy & Compliance:** Eliminate human data entry errors to ensure accurate customs declarations and avoid fines.
- **Increase Operational Efficiency:** Free up logistics operators from manual data entry to focus on high-value exception handling and customer service.

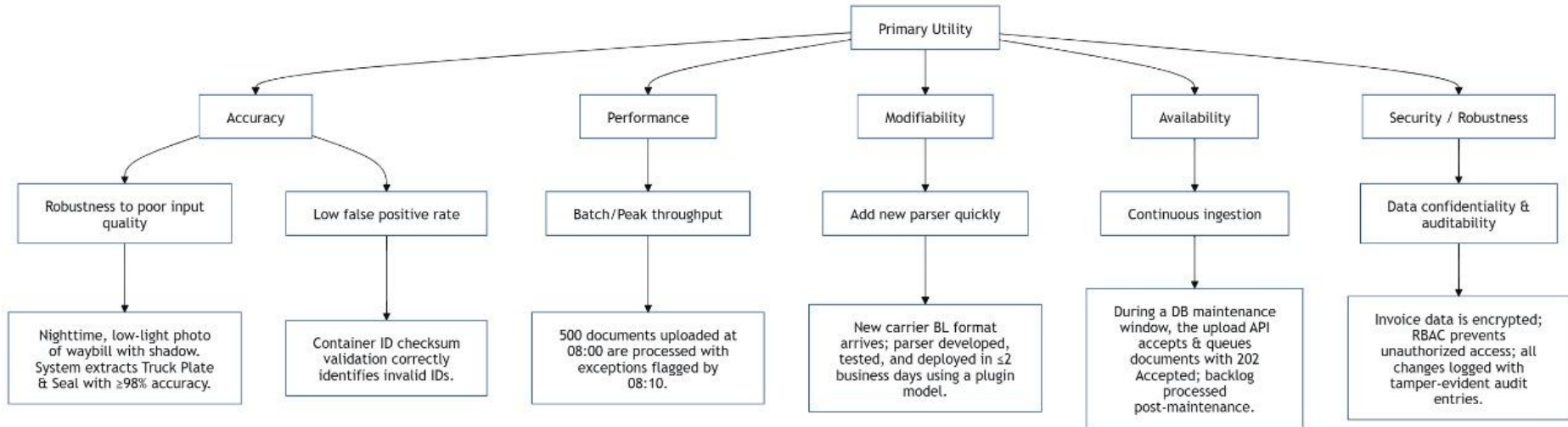
FUNCTIONAL REQUIREMENTS:

FR No.	Description
FR1 — — —	Accept multi-page documents (PDF, JPG, TIFF) from web upload, mobile app, email ingest, and carrier portals.
FR2	Automatic per-page classification into document types (e.g., Commercial Invoice, Bill of Lading, Packing List, Waybill).
FR3	Extract a predefined schema of fields per type (Container ID, BL No., HS Code, Shipper/Consignee, Truck Plate, Seal No., Net/Gross weights).
FR4	Provide a Verification UI for human review/confirmation of low-confidence or exception fields, with quick edit, versioning, and audit trail.
FR5	Expose REST APIs for programmatic submission, status polling, and retrieval of structured results (JSON/CSV).
FR6	Push final, verified records to external systems: internal ERP (JSON/XML) and national customs (EDIFACT/XML) via EDI Gateway.
FR7	Provide role-based access and audit logs for all document operations (ingest, review, export).

NON-FUNCTIONAL REQUIREMENTS:

NFR No.	Description / Measurable Target
NFR1(Accuracy)	Target: >95% accuracy on critical fields (Container ID, BL No.). Measure: periodic test set evaluation and production drift monitoring.
NFR2 (Performance)	Target: Average shipment (200 pages) processed under 10 minutes; bursts (500 docs) finished under 10 minutes with no more than 5% SLA violation.
NFR 3 (Availability)	Target: 99.9% uptime for upload API; ingestion must accept requests during partial outages.
NFR4 (Modifiability)	Target: New document parser added & deployed in ~3 business days with no downtime and regression tests.
NFR5 Interoperability)	Must interoperate with legacy EDIFACT customs systems; support both JSON and EDIFACT export configurations.
NFR6 (Security & Privacy)	Data encrypted in transit (TLS1.2+) and at rest (AES-256); RBAC for UI/API; full audit trail and retention policy.
NFR7 (Scalability)	Horizontal scaling for OCR workers; autoscaling policies to 10x baseline for peak windows.
NFR8 (Observability)	Centralized logging, metrics (latency, throughput, field confidence), and alerting for model drift and service degradation.

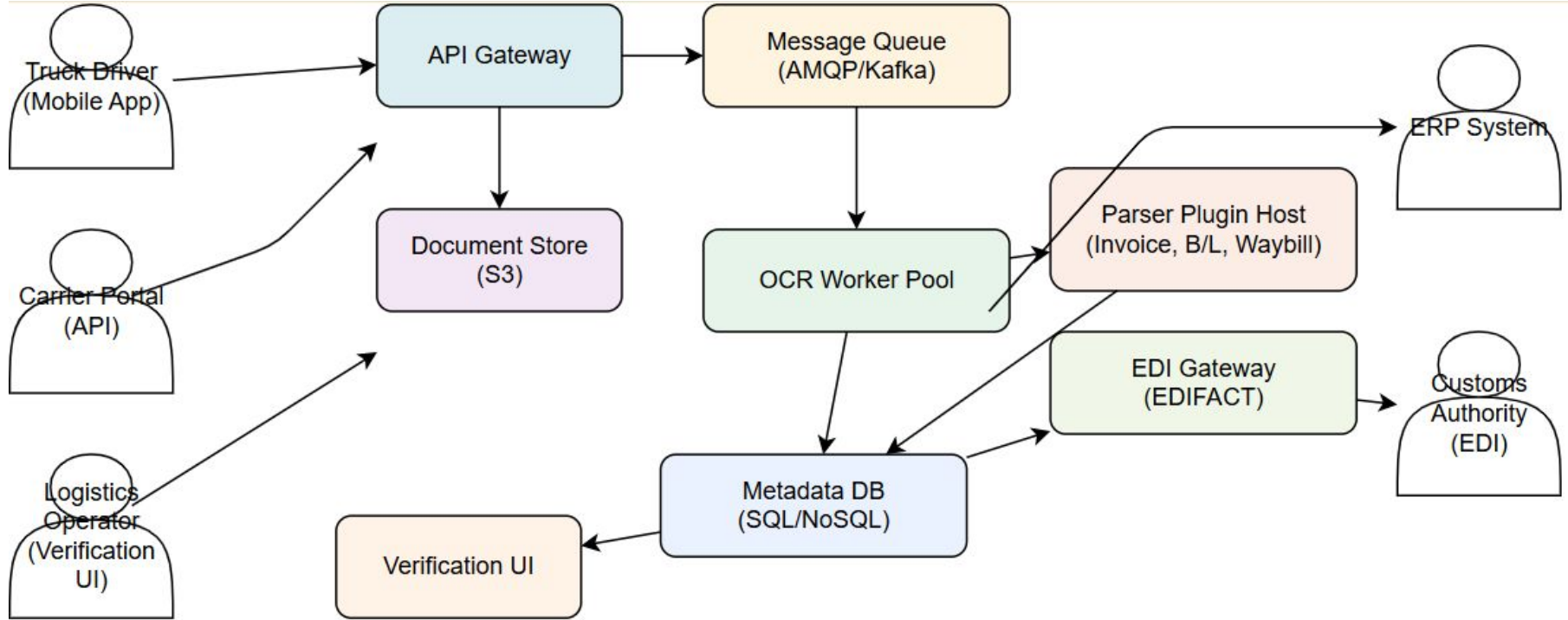
UTILITY TREE OF ARCHITECTURALLY SIGNIFICANT REQUIREMENTS (ASR)



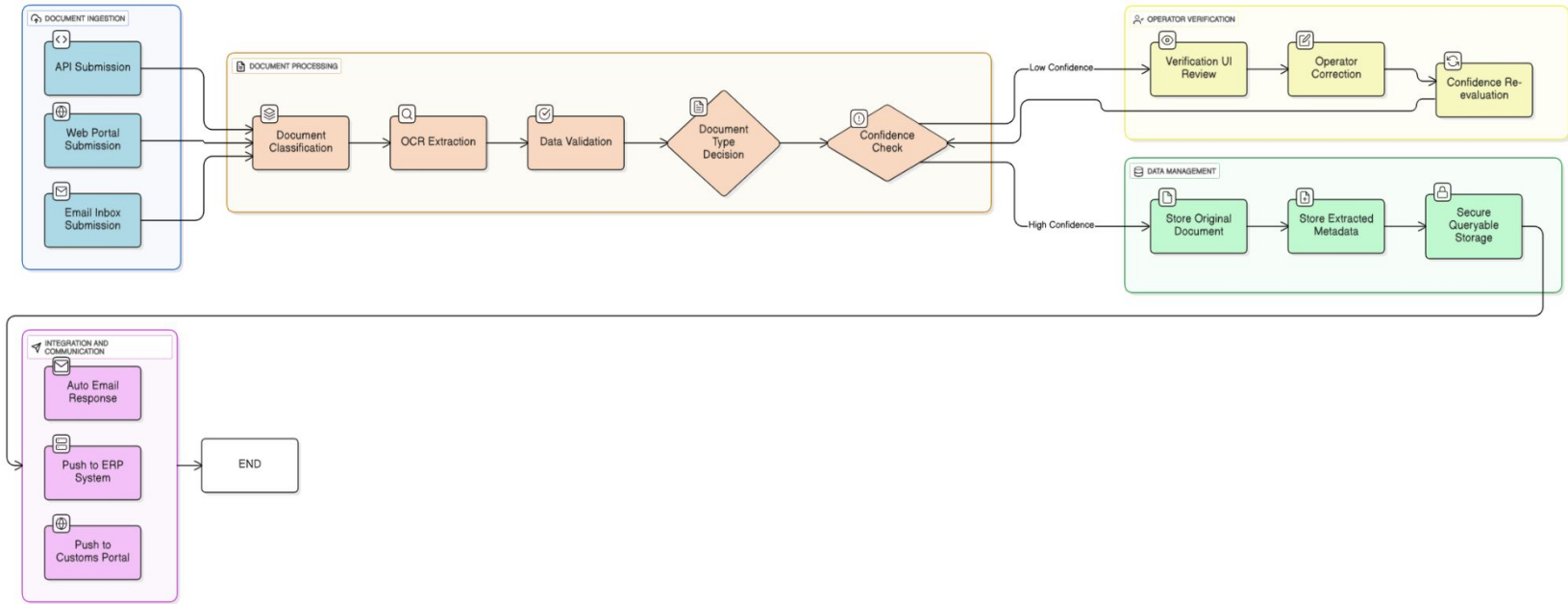
TACTICS USED TO ACHIEVE THE TOP 5 ASRs

Quality Attribute	Scenario (Attribute Refinement)	Tactics
High Throughput (500 docs <10 min)	Message Queue (durable, multi-AZ); Autoscaling OCR Worker Pool; Parallelism & batching	The queue decouples the API from CPU-heavy OCR; workers scale horizontally and process in parallel. Batching reduces I/O.
Rapid Modifiability	Plugin Parser Architecture + Semantic Parser Interface (Document Parser); CI pipeline & contract tests	Isolates parser logic; a new parser implements the interface and passes contract tests, allowing independent deployment.
Accuracy on Poor Scans	Preprocessing Pipeline (de-skew, denoise, contrast), Ensemble OCR + Domain Models, Post Processing (rules & checksum)	Preprocessing normalize input; the ensemble increases robustness; post processing validations (regex, checksums) catch errors.
Continuous Ingestion / Availability	Asynchronous Messaging, API only enqueues; Redundant queue (multi-region); Circuit breakers & retries	The API stays up by offloading to the queue; the system can accept traffic even when downstream components are unavailable.
Interoperability (EDIFACT)	Adapter Pattern + EDI Orchestrator; Mapping DSL / template engine	Translators isolate EDIFACT mapping rules; the orchestrator handles retries, acknowledgments, and protocol idiosyncrasies.

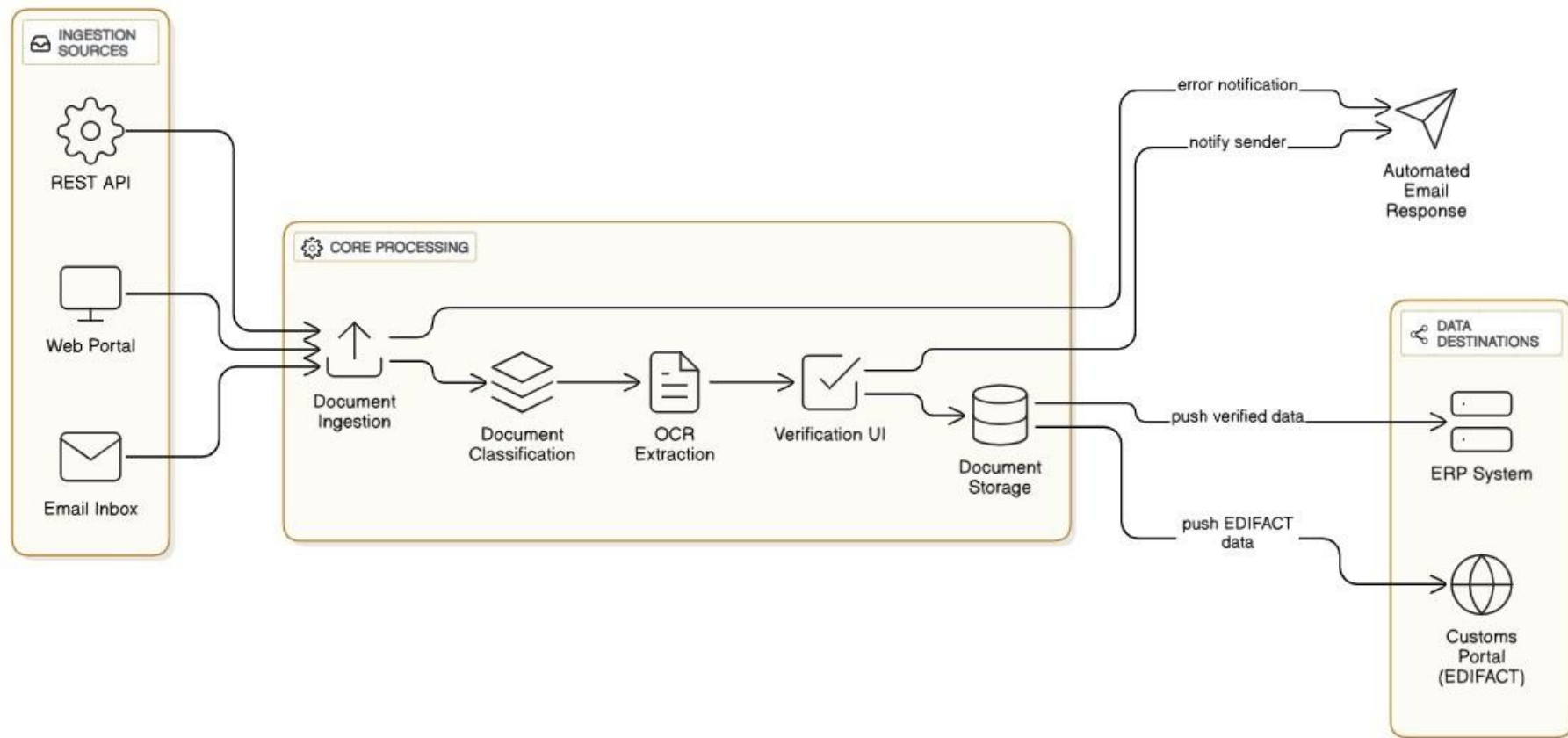
CONTEXT DIAGRAM



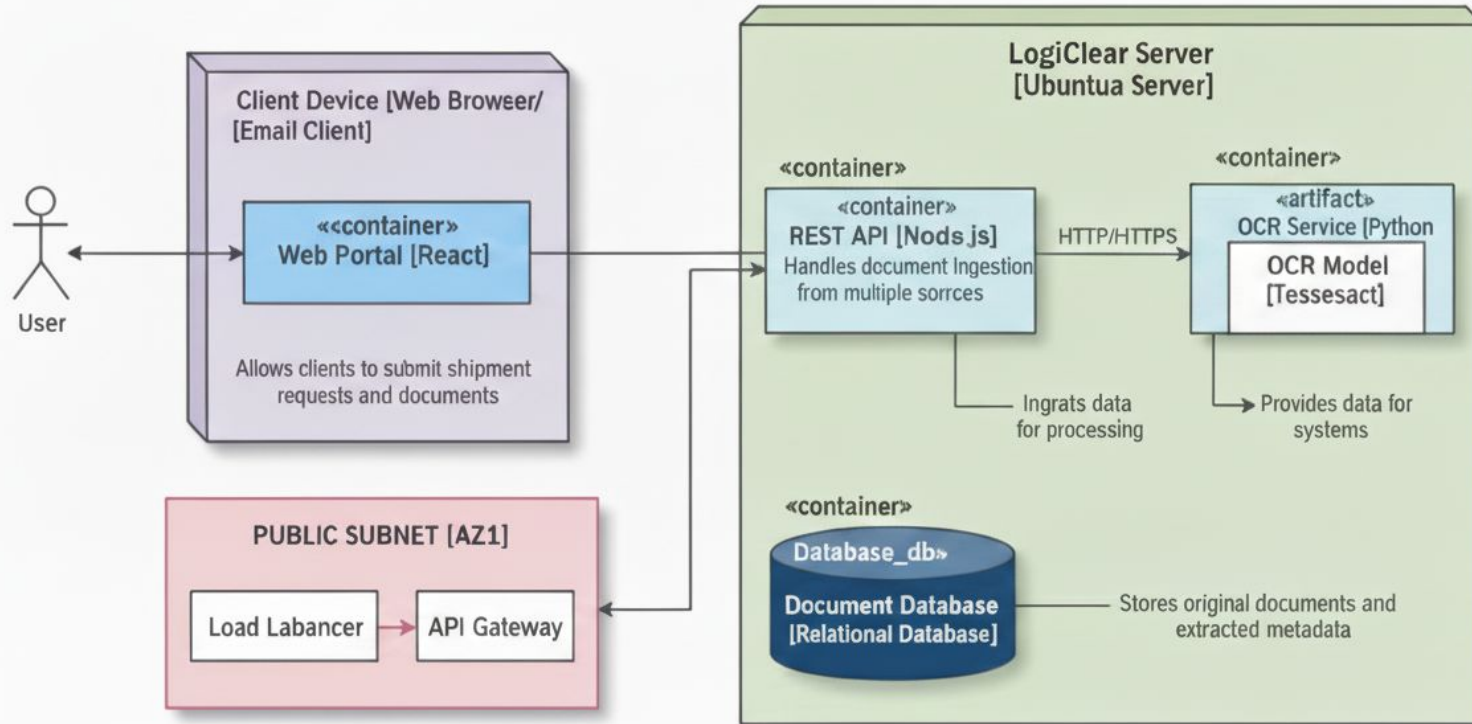
MODULE DECOMPOSITION



COMPONENT CONNECTION DIAGRAM



DEPLOYMENT DIAGRAM



System Flow: Step-by-Step

1. **Document Ingestion:** A user or an automated system submits a document through one of the Ingestion Sources:

- **Web Portal / Mobile App:** A user uploads a file (e.g., a driver photographing a waybill).
- **Email Inbox:** The system automatically pulls attachments from a dedicated email address.
- ~~REST API:~~ A partner system programmatically submits a document.

Architecturally, the request hits an API Gateway. The system immediately saves the raw file to a document store (like S3) and places a "new document" message onto a Message Queue. It then returns a 202 Accepted response. This makes the ingestion process extremely fast and reliable, as it doesn't wait for the full processing to complete.

2. **Core Processing:** A pool of backend workers listens to the queue. When a new message appears, a worker picks it up and begins a multi-stage process:

- **Document Classification:** The system first analyzes the document to determine its type (e.g., "Commercial Invoice," "Bill of Lading," "Truck Waybill").
- **OCR Extraction:** Based on the document type, a specific parser plugin is selected. An OCR (Optical Character Recognition) engine extracts all text and data.
- **Data Validation & Confidence Scoring:** The system validates the extracted data (e.g., checking container ID checksums) and calculates a confidence score for each field. This score represents how certain the AI is about the accuracy of the extracted data.

3. The Confidence Check (Automated Decision): This is the critical branching point shown in the flowchart.

- **If High Confidence:** (e.g., all field scores > 90%), the process is fully automated. The system proceeds directly to Step 5.
- **If Low Confidence:** (e.g., a blurry "Seal Number" has a score of 82%), the document is flagged. The extracted data is saved, and a task is created for a human to review.

4. Human Verification (Human-in-the-Loop): When a document is flagged, it appears in a queue in the Verification UI.

- A human operator reviews the original document alongside the extracted data.
- They can quickly correct any low-confidence fields (like fixing the Seal Number).
- Upon approval, the action is logged for a full audit trail, and the now-verified data is sent forward in the process.

5. Data Storage & Integration: Once the data is either auto-approved or manually verified, two things happen:

- **Data Management:** The final, clean data and its metadata are stored in a database for analytics and querying. The original document is archived.
- **Integration & Communication:** The verified data is sent to a final "export" queue. From there, specialized components push it to the required Data Destinations:
 - An EDI Gateway transforms the data into the EDIFACT format and transmits it to the Customs Portal.
 - Another service pushes the data as JSON/XML to the internal ERP System.
 - Finally, a Notification Service sends an automated email or SMS response to the original sender or relevant stakeholders, confirming the process is complete.

KEY LEARNINGS

- **Architecture is Dictated by Trade-offs, Not Perfection.** There is no single "best" architecture; every design choice is a compromise. The architect's primary role is to understand the business drivers, identify the most critical quality attributes (like performance vs. consistency), and make conscious, justifiable trade-offs that align with those priorities.
- A context diagram aligns stakeholders, a component diagram guides developers, and a deployment diagram informs the operations team. The ability to create a shared understanding across diverse groups—translating business needs into technical strategy and back again—is more crucial than mastering any single technology. **Communication is the Architect's Most Powerful Tool**
- The analysis of the system in different views (Module decomposition, utility trees, component connection etc.) can be templated and be reused for other similar systems. This helps in deriving more cost vs benefit of different architecture approaches for similar problems in the future.
- **Design for What You Can't See:** Observability. It's not enough for the system to work correctly, we also need to see how it works in real time. That means adding logging, metrics, and tracing right from the start, not later as a patch, which reflect a good architecture design.

Thank you!